RELATÓRIO DE AUDITORIA TÉCNICA COMPLETA — OPEN TMS

Avaliação Arquitetural, Funcional, Estrutural e de Segurança para Preparação de um TMS SaaS Brasileiro

---

Identificação do Projeto: Open TMS (Open Transport Management System)
Repositório Oficial: `https://github.com/DominicFinn/open_tms.git`
Autor Original: Dominic Finn
Licença: MIT (Copyright 2025 Dominic Finn)
Data da Auditoria: 10 de Novembro de 2026
Auditor Responsável: Equipe de Engenharia / Antigravity Agentic AI
Ambiente Auditado: Linux Ubuntu 24.04.1 LTS (Kernel 7.0.0-42-generic x86_64), Node.js v24.22.1, npm 11.19.1, Docker 29.1.3, Docker Compose 2.40.3
Commit Auditado: `ed831c2b9f77721cc5c9d87c49f9aa3bde8124a0` (Branch `main`, árvore de trabalho limpa)

SUMÁRIO EXECUTIVO

A presente auditoria técnica foi realizada sobre a base de código do projeto de código aberto **Open TMS**, com o objetivo de analisar sua viabilidade, arquitetura, segurança, integridade relacional e prontidão funcional para servir de fundação a uma plataforma comercial **SaaS de TMS (Transport Management System) voltada ao mercado brasileiro**.

Conclusão de Viabilidade Técnica:

O projeto Open TMS é TECNICAMENTE VIÁVEL como base aceleradora para o SaaS

brasileiro, proporcionando uma economia estimada entre **9 e 14 meses de desenvolvimento** na implementação dos motores centrais de logística (despacho, roteamento, conferência de carga, topologia de armazém WMS, esteira de exceções logísticas e consolidação de fretes LTL).

Contudo, **o sistema NÃO está pronto para implantação ou comercialização imediata**. A base tecnológica apresenta quatro ordens de desafios críticos que exigem intervenção estrutural prévia:

1. **Quatro vulnerabilidades críticas de segurança (P0)**, incluindo SSRF com exfiltração em webhooks, ingestão EDI desprotegida e ausência de Row Level Security (RLS) no PostgreSQL;
2. **Incompatibilidade regulatória total com o Brasil**, sem qualquer suporte a documentos fiscais obrigatórios (CT-e, MDF-e, NF-e), CIOT, Vale-Pedágio, CNH, CPF/CNPJ ou Placas Mercosul;
3. **Inexistência de camada comercial SaaS**, necessitando criação de controle de planos, quotas de uso, faturamento recorrente nacional e painel Super-Admin de plataforma;
4. **Vulnerabilidades de dependências (CVEs)** nos pacotes npm, exigindo atualização cautelosa de bibliotecas chave.

---

1. INTRODUÇÃO E OBJETIVO DA AUDITORIA

O setor de transporte rodoviário de cargas no Brasil possui particularidades regulatórias, fiscais e operacionais que o diferenciam substancialmente dos mercados norte-americano e europeu. A proposta de criar uma plataforma SaaS acessível a **Transportadores Autônomos de Cargas (TACs)**, pequenos transportadores e empresas com frotas próprias ou agregadas requer um software robusto, escalável, seguro e em estrita conformidade com a legislação nacional (ANTT, SEFAZ, Receita Federal e LGPD).

O objetivo desta auditoria é dissecar o código-fonte real do Open TMS, avaliando:

- A viabilidade da arquitetura de software para múltiplas organizações em regime SaaS;
- A integridade e segurança do banco de dados relacional e a eficácia do isolamento entre empresas;
- A postura de segurança cibernética contra vetores clássicos de ataque (OWASP Top 10);
- O inventário real de funcionalidades prontas versus funcionalidades parciais ou inexistentes;
- As adaptações indispensáveis para atendimento à legislação e operação logística do Brasil;
- A conformidade das licenças de software e vulnerabilidades conhecidas em dependências;
- A formulação de uma matriz de riscos priorizada e um roadmap técnico de evolução em 12 etapas.

---

2. METODOLOGIA E AMBIENTE UTILIZADO

2.1. Metodologia de Investigação

A auditoria fundamentou-se em inspeção estática e dinâmica de código, análise de dependências e execução de testes em ambiente local isolado, orientando-se pelas seguintes premissas:

- **Inspeção de Código-Fonte Real:** Análise linha a linha das rotas, middleware, comandos, serviços de domínio e schemas de banco, recusando qualquer suposição baseada unicamente no `README.md` ou documentações promocionais.
- **Não Destrutividade:** Não foram feitas alterações funcionais, refatorações, migrações destrutivas ou inserção de credenciais reais no repositório.
- **Rastreabilidade e Evidências:** Todo achado técnico possui correspondência direta com arquivos, linhas de código e testes automatizados executados.

- **Alinhamento com Normas ABNT:** Estruturação lógica e padronizada de relatório técnico de software (ABNT NBR 10719).

2.2. Registro Formal do Ambiente de Execução

O ambiente local onde a auditoria foi conduzida possui as seguintes especificações verificadas:

- **Sistema Operacional:** Linux Ubuntu 24.04.1 LTS (Kernel: `7.0.0-42-generic #42~24.04.1-Ubuntu SMP x86_64`)
- **Node.js:** Versão `v24.22.1`
- **npm:** Versão `11.19.1`
- **Git:** Versão `2.43.0`
- **Docker Engine:** Versão `29.1.3`
- **Docker Compose:** Versão `2.40.3`
- **Memória RAM:** 7,7 GiB total (disponibilidade média: 3,2 GiB)
- **Armazenamento:** 234 GiB total (180 GiB livres na partição `/`)

2.3. Registro Formal de Clonagem

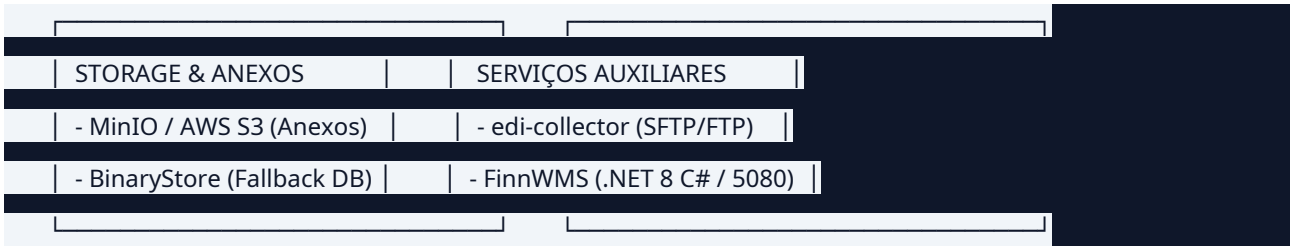
- **URL do Repositório:** `https://github.com/DominicFinn/open_tms.git`
- **Diretório de Trabalho:** `/home/luisfvmartins/Projetos/Open-TMS`
- **Branch Ativa:** `main` (atualizada com `origin/main`)
- **Commit Analisado:** `ed831c2b9f77721cc5c9d87c49f9aa3bde8124a0`
- **Data da Clonagem:** 10 de Novembro de 2026
- **Estado do Repositório:** Árvore de trabalho limpa (`working tree clean`), sem modificações locais não rastreadas.

---

3. ARQUITETURA ATUAL DO SISTEMA

O Open TMS é estruturado primariamente como um **monorepo modular** gerenciado via npm workspaces, combinando um backend em TypeScript/Fastify, um frontend SPA em React 18, processos assíncronos via workers e um microsserviço satélite em .NET para rotas de WMS.





3.1. Componentes e Responsabilidades

1. **Frontend (`frontend/`):**
    - Construído com React 18, TypeScript, Vite, Tailwind CSS e componentes Radix UI.
    - Apresenta mais de 80 páginas funcionais cobrindo embarques, pedidos, rotas, consolidação, tabelas tarifárias, relatórios financeiros e painéis de cais.
    - Empacotado em bundle único de 2,5 MB com Tailwind pré-compilado.
2. **Backend Fastify (`backend/`):**
    - Construído em Fastify 4.28 com TypeScript compilado para ES Modules.
    - Estruturado segundo princípios de Clean Architecture e CQRS: rotas finas delegam mutações para o `CommandBus`, que grava a entidade e o evento de domínio no mesmo bloco transacional (`BaseCommandHandler`).
    - Leitura de alto volume otimizada através de tabelas desnormalizadas (`*ReadModel`).
3. **Processador Assíncrono (`backend/src/worker.ts`):**
    - Processo Node.js desacoplado, projetado para rodar em container isolado sem expor portas HTTP.
    - Consome filas do `pg-boss` para atualizar projeções de leitura, disparar webhooks, monitorar atrasos de rotas (ETA Monitor), avaliar SLAs e executar triagem inteligente de incidentes logísticos via Anthropic Claude.
4. **Microsserviço FinnWMS (`services/wms/`):**
    - Implementado em .NET 8 (C#) com Dapper e migrations SQL brutas.
    - Atende rotas operacionais de depósito em cais com suporte a Row Level Security nativo no schema `wms`.
5. **EDI Collector (`edi-collector/`):**

- Daemon dedicado para polling de servidores remotos de clientes via SFTP/FTP, coletando arquivos de intercâmbio eletrônico de dados (EDI) e repassando à API.

---

4. AUDITORIA DO BANCO DE DADOS E MULTI-TENANCY

4.1. Estrutura do Prisma Schema

O banco de dados é modelado utilizando o recurso de múltiplos arquivos de schema do Prisma 5.22, divididos em sete domínios:

- `core.prisma`: Organização, Usuários, Papéis, Permissões, Auditoria, Anexos, Campos Customizados e Eventos de Domínio;
- `tms.prisma`: Embarques (`Shipment`), Pedidos (`Order`), Rotas (`Lane`), Consolidação, Transportadoras, Dispositivos IoT, Cadeia Fria e Ocorrências;
- `finance.prisma`: Faturas (`Invoice`), Contas a Pagar (`CarrierInvoice`), Cotações (`Quote`), Encargos (`Charge`), Pagamentos e Comissões;
- `wms.prisma`: Armazéns, Zonas, Posições (`Bin`), Recebimento, Putaway, Ondas, Separação, Embalagem e Despacho;
- `inventory.prisma`: Registro e Transações de Estoque, Observações e Alocações;
- `quality.prisma`: Relatórios CAPA, Checklists e Auditorias de Procedimento Padrão (SOP);
- `config.prisma`: Parâmetros do sistema.

4.2. Avaliação de Multi-Tenancy e Isolamento

A plataforma utiliza o modelo de **banco de dados compartilhado com segregação lógica por coluna** (`orgId` em cada tabela).

Pontos Positivos Identificados:

- Todas as entidades operacionais principais (`Shipment`, `Order`, `Customer`, `Carrier`, `Lane`,

`Location`, `Invoice`, `Vehicle`, `Driver`) possuem coluna `orgId String NOT NULL` acompanhada de índices indexados `@@index([orgId])`.

- O framework CQRS utiliza tabelas de leitura (`*ReadModel`) com isolamento estrito por `orgId`.
- As rotas da API autenticada passam pelo hook `registerStrictOrgScope`, que extrai o tenant do JWT e rejeita requisições sem contexto com código HTTP 401.

Lacunas Críticas Identificadas:

1. **Ausência de Row-Level Security (RLS) no PostgreSQL (Achado P0-TEN-01):**
    - O isolamento depende inteiramente da integridade do código da aplicação (Prisma). Não há nenhuma política de segurança nativa no PostgreSQL para impedir que uma query mal formatada ou executada via Dapper leia registros de outras empresas.
2. **Ausência de Chaves Estrangeiras Compostas (Achado P0-TEN-02):**
    - As constraints de banco vinculam apenas o ID simples. Nada impede que uma linha pertença à Empresa A e aponte para um cliente ou veículo da Empresa B, abrindo brecha para relacionamentos cruzados caso a validação do backend falhe.
3. **Rigidez de Usuário Único por Empresa (Achado P1-SAA-02):**
    - O modelo `User` possui a restrição `email String @unique` e um campo `organizationId String` direto. Um operador ou motorista não pode usar o mesmo endereço de e-mail em duas organizações simultaneamente.

---

5. AUDITORIA DE SEGURANÇA DA APLICAÇÃO

A auditoria de segurança avaliou as defesas da aplicação contra vulnerabilidades do OWASP Top 10 e práticas de segurança corporativa.

5.1. Autenticação e Gestão de Credenciais

- **Algoritmo Inseguro de Hash de Senha (Achado P1-SEC-03):** O backend deriva senhas utilizando HMAC-SHA256 simples de 1 iteração (`backend/src/services/AuthService.ts:227-231`). Essa abordagem viola normas modernas (OWASP) e permite quebra rápida por força bruta em caso de vazamento da base.
- **Sessões Stateless Sem Revogação (Achado P1-SEC-04):** Os tokens de acesso possuem validade estática de 12 horas. O logout na rota `POST /api/v1/auth/logout` é um no-op que não invalida o token no servidor.
- **Recuperação de Senha Inoperante:** A rota `/api/v1/auth/forgot-password` é um stub que apenas imprime um log informativo, orientando o usuário a procurar o administrador.

5.2. Autorização e Controle de Acesso (RBAC)

- O sistema possui controle de acesso baseado em papéis (`Role`) e permissões granulares (`users:read`, `shipments:write`, etc.) armazenadas em array JSON.
- A maior parte dos endpoints internos valida as permissões corretamente via middleware `requirePermission`.

5.3. Vulnerabilidades Web Identificadas

1. **Server-Side Request Forgery - SSRF (Achado P0-SEC-01):**
    - Localizado em `CustomerWebhookDeliveryService.ts:83` e `CallWebhookSkill.ts:44`.
    - Permite que um usuário informe endpoints internos (`169.254.169.254`, `127.0.0.1`) e consulte até 4.000 caracteres da resposta através da API de histórico de entregas.
2. **Ingestão EDI Desprotegida (Achado P0-SEC-02):**
    - Rotas `/api/v1/edi/*` estão registradas como públicas sem validação de HMAC ou API Key, permitindo que qualquer agente anônimo injete transações de frete caso conheça o ID do parceiro.
3. **Stored Cross-Site Scripting - XSS (Achado P1-SEC-05):**

- Localizado em `VNextEmailTemplates.tsx:378`.
- Renderização de código HTML sem higienização via `dangerouslySetInnerHTML`, expondo o `auth_token` gravado no `localStorage`.

4. **Ausência de Rate Limiting Global (Achado P1-SEC-06):**
    - Endpoints de login e recuperação de senha desprotegidos contra negação de serviço e ataques de força bruta em massa.

---

6. COMPATIBILIDADE COM O MERCADO BRASILEIRO

O código do Open TMS foi desenvolvido sob premissas estritas da logística dos Estados Unidos. A tabela a seguir compara o estado atual com os requisitos obrigatórios do mercado brasileiro:

Requisito do Brasil	Situação no Open TMS	Gravidade	Impacto no Lançamento
Identificação Pessoal / Fiscal	Inexistente. Apenas `taxId` livre ("VAT / EIN"). Sem CPF, CNPJ ou suporte ao novo CNPJ alfanumérico.	**P1 - Alto**	Bloqueia cadastros formais de clientes, motoristas e transportadoras.
Endereçamento Nacional	Parcial. Endereço padrão EUA (`address1`, `state`, `postalCode`). Sem bairro, sem código IBGE de município e sem busca automática de CEP.	**P1 - Alto**	Inviabiliza a emissão de documentos fiscais eletrônicos (que exigem código IBGE).

Requisito do Brasil	Situação no Open TMS	Gravidade	Impacto no Lançamento
Documentação Veicular	Inexistente. Placa é string livre sem validação. Sem RENAVAM, Chassi ou suporte ao padrão Mercosul (`AAA0A00`).	**P1 - Alto**	Impede registro fidedigno de frotas e fiscalização em postos fiscais.
Documentos de Motorista	Inexistente. Sem CNH, sem categoria (C, D, E), sem vencimento e sem exames toxicológicos.	**P1 - Alto**	Risco de não conformidade trabalhista e regulatória (ANTT).
Tipos de Frota	Rígido. Motorista só existe subordinado a uma `Carrier`. Não suporta TACs autônomos diretos nem agregados.	**P1 - Alto**	Impede o atendimento ao público prioritário inicial (motoristas autônomos).
Documentos Fiscais (CT-e / MDF-e)	Inexistente. Sistema gera apenas Bill of Lading (BOL), documento sem valor fiscal no Brasil.	**P1 - Alto**	Bloqueio comercial legal. Cargas sem CT-e e MDF-e são apreendidas pela fiscalização.
Conexão com NF-e	Inexistente. Pedidos não capturam chave de acesso de 44 dígitos da NF-e nem realizam leitura de XML.	**P1 - Alto**	Transportadoras brasileiras transportam mercadorias amparadas por NF-e de terceiros.
CIOT e Vale-Pedágio	Inexistente. Sem integração com IPEFs para geração de CIOT ou comprovante de Vale-Pedágio obrigatório.	**P1 - Alto**	Multas severas aplicadas pela ANTT por descumprimento da Lei 10.209/2001.
Padrão de EDI	Incompatível. Sistema suporta	**P2 - Médio**	Grandes embarcadores

Requisito do Brasil	Situação no Open TMS	Gravidade	Impacto no Lançamento
Nacional	apenas ANSI X12 norte-americano. O padrão brasileiro é PROCEDA (NOTFIS, OCOREN).		brasileiros exigem integração via layout PROCEDA.
Meios de Pagamento	Incompatível. Moeda padrão USD; liquidação financeira manual. Sem PIX ou Boleto Bancário.	**P1 - Alto**	Faturamento inviabilizado para clientes e motoristas no Brasil.

---

7. AUDITORIA DE DEPENDÊNCIAS E LICENÇAS

7.1. Análise da Licença de Software

O repositório oficial do Open TMS é distribuído sob os termos da **Licença MIT**, registrada no arquivo LICENCE.md em nome de **Dominic Finn (2025)**:

> **Implicações Comerciais:**
>
> 1. A licença MIT é uma licença de software livre permissiva que **autoriza expressamente o uso comercial, modificação, distribuição, sublicenciamento e fechamento de código derivado**, sem exigir o compartilhamento de código-fonte de modificações proprietárias.
> 2. **Obrigação Legal Inviolável:** A licença impõe a condição jurídica mandatória de que o aviso original de direitos autorais (*Copyright (c) 2025 Dominic Finn*) e o texto da permissão da licença MIT sejam mantidos e reproduzidos em todas as cópias ou partes substanciais do software.
> 3. **Conformidade das Dependências:** A auditoria via `license-checker` confirmou que

> 100% das dependências de produção utilizam licenças compatíveis (MIT: 710 pacotes, Apache-2.0: 61, ISC: 55, BSD: 36, BlueOak: 8, 0BSD: 2). Não foi detectada nenhuma dependência com licença copyleft restritiva (como GPL ou AGPL) no código compilado de produção.

7.2. Auditoria de Vulnerabilidades em Dependências

A execução do comando `npm audit` revelou 45 vulnerabilidades conhecidas no grafo de dependências:

- **Críticas:** 4
- **Altas:** 10
- **Moderadas:** 28
- **Baixas:** 3

Dentre as vulnerabilidades de alto impacto técnico, ressaltam-se:

1. `@fastify/reply-from <= 12.6.1` (**Crítica — GHSA-gwhp-pf74-vj37**): Permite manipulação e remoção de cabeçalhos proxy de segurança adicionados pelo backend através de abuso do cabeçalho `Connection`.
2. `@fastify/busboy < 3.2.1` (**Alta — GHSA-x8mw-p69m-v3mx**): Negação de serviço (DoS) via injeção de cabeçalhos de partes multipart nomeados com propriedades de protótipo.
3. `undici < 6.27.0` (**Alta — GHSA-vxpw-j846-p89q**): Evasão de contagem de fragmentos em clientes WebSocket levando a esgotamento de memória.
4. `zod < 4.6.6` (**Alta — GHSA-mrhx-c7xc-m3w5**): Esgotamento de memória ao validar coleções excessivamente aninhadas ou de tamanho anômalo.

---

8. RESULTADOS DE EXECUÇÃO E TESTES AUTOMATIZADOS

O projeto foi submetido a execução completa em ambiente local sem qualquer alteração prévia de código.

8.1. Compilação e Build

- **Pacotes Compartilhados (`packages/shared`):** Compilação TypeScript executada com sucesso (`tsc -p tsconfig.json`).
- **Prisma Client:** Geração do cliente concluída com êxito em 7.84s a partir dos 7 schemas (`backend/prisma/schema/*.prisma`).
- **Backend (`@open-tms/backend`):** Compilação TypeScript de 935 arquivos executada com sucesso sem nenhum erro de tipagem (`tsc -p tsconfig.json`).
- **Frontend (`@open-tms/frontend`):** Compilação e bundling via Vite concluídos com sucesso em 46.26s (`3.485 módulos transformados`), gerando artefatos estáticos prontos para produção.

8.2. Linters Arquiteturais Próprios

O projeto possui dois linters arquiteturais desenvolvidos sob medida para garantir integridade estrutural:

1. **Module Boundaries Linter (`npm run lint:boundaries`):**
    - Resultado: **Aprovado**. Analisou 935 arquivos com 5 exceções justificadas na linha de base e 24 acessos cruzados tolerados entre tabelas.
2. **Tenancy Linter (`npm run lint:tenancy`):**
    - Resultado: **Aprovado**. Verificou a presença de escopo de tenant em repositórios e comandos, sem violações registradas.

8.3. Execução da Suíte de Testes Automatizados

A execução dos testes existentes no repositório apresentou resultados excepcionais de aprovação:



Backend	251	2.595	0	161,45 s
Frontend	8	61	0	111,65 s
TOTAL	259	2.656	0 (100%)	273,10 s

Alertas e Observações Importantes:

- **100% de Aprovação:** Todos os 2.656 testes existentes foram aprovados com sucesso.
- **Desequilíbrio de Cobertura:** Enquanto o backend possui 251 suítes de teste (cobrindo extensivamente CQRS, comandos, isolamento de tenant, regras de negócio e integrações), o frontend possui apenas 8 suítes com 61 testes focados quase exclusivamente em utilitários e providers, deixando telas e formulários sem testes automatizados.
- **Vazamento de Handles no Jest:** Os scripts do Jest exigem a flag `--forceExit` porque conexões com o banco ou timers assíncronos não são limpos adequadamente no teardown dos testes (`Active timers can cause this, ensure that .unref() was called on them`).

---

9. QUADRO CONSOLIDADO DE RISCOS E ACHADOS

Abaixo estão sumarizados todos os 20 achados técnicos identificados na auditoria, ordenados rigorosamente por nível de criticidade:

ID	Categoria	Criticidade	Descrição Sucinta	Localização no Código	Complexidade de Ajuste
P0-SEC-01	Segurança	**P0 - Crítico**	SSRF com exfiltração de resposta HTTP em webhooks e	`CustomerWebhookDeliveryService.ts:83`	Média

ID	Categoria	Criticidade	Descrição Sucinta	Localização no Código	Complexidade de Ajuste
			agente		
P0-SEC-02	Segurança	**P0 - Crítico**	Rotas EDI Inbound sem autenticação ou validação de assinatura	`backend/src/routes/modules/tms.ts:82`	Média
P0-TEN-01	Multi-tenancy	**P0 - Crítico**	Ausência de PostgreSQL RLS (isolamento depende 100% do Prisma)	`backend/prisma/migrations/`	Alta
P0-TEN-02	Multi-tenancy	**P0 - Crítico**	Falta de Foreign Keys compostas com `orgId` (risco de relação cruzada)	`backend/prisma/schema/tms.prisma:4-50`	Média
P1-SEC-03	Segurança	**P1 - Alto**	Hash de senhas inseguro (HMAC-SHA256 simples sem fator de custo KDF)	`AuthService.ts:227`, `CustomerAuthService.ts:226`	Baixa
P1-SEC-04	Segurança	**P1 - Alto**	JWT stateless de 12 horas sem revogação no servidor e logout	`AuthService.ts:16`, `routes/auth.ts:120`	Média

ID	Categoria	Criticidade	Descrição Sucinta	Localização no Código	Complexidade de Ajuste
			fictício		
P1-SEC-05	Segurança	**P1 - Alto**	Tokens em localStorage e Stored XSS em preview de templates de e-mail	authFetch.ts:28, VNextEmailTemplates.tsx:378	Baixa
P1-SEC-06	Segurança	**P1 - Alto**	Ausência de rate limiting global nos endpoints de autenticação e API	backend/src/index.ts	Baixa
P1-BRA-01	Adaptação BR	**P1 - Alto**	Cadastros sem campos brasileiros (CPF, CNPJ, CEP, IBGE, Placa Mercosul)	schema/core.prisma:39, schema/tms.prisma:488	Média
P1-BRA-02	Compliance BR	**P1 - Alto**	Ausência total de módulo fiscal brasileiro (CT-e, MDF-e, CIOT, Pedágio)	DocumentGenerationService.ts	Alta
P1-BRA-03	Operação BR	**P1 - Alto**	Falta de suporte a TAC, agregados, separação	schema/tms.prisma:481-516	Alta

ID	Categoria	Criticidade	Descrição Sucinta	Localização no Código	Complexidade de Ajuste
			cavalo/carreta e acertos		
P1-SAA-01	SaaS Comercial	**P1 - Alto**	Inexistência de gestão comercial SaaS (planos, quotas, faturamento)	backend/prisma/schema/	Alta
P1-SAA-02	Multi-tenancy	**P1 - Alto**	Usuário restrito a 1 organização (e-mail único global no sistema)	schema/core.prisma:404-416	Média-Alta
P1-DEP-01	Dependências	**P1 - Alto**	14 vulnerabilidades críticas e altas em dependências npm	package-lock.json, audit_results.json	Média
P2-ARQ-01	Desempenho	**P2 - Médio**	Banco compartilhado entre OLTP, Event Store e Filas pg-boss (bloat)	PgBossQueueAdapter.ts, worker.ts	Média
P2-ARQ-02	Arquitetura	**P2 - Médio**	Desalinhamento da stack mista .NET C#	services/wms/, docs/adr/0002.md	Média

ID	Categoria	Criticidade	Descrição Sucinta	Localização no Código	Complexidade de Ajuste
			(WMS) e diretório legado auth		
P2-TST-01	Qualidade	**P2 - Médio**	Desequilíbrio na cobertura de testes (frontend com 8 testes) e forceExit	frontend/src/__tests__/, package.json:22	Média
P2-WOR-01	IA & Worker	**P2 - Médio**	Incompatibilidade multi-tenant do Agente de IA nos workers	backend/src/worker.ts:149-165	Baixa-Média
P3-DES-01	Performance	**P3 - Baixo**	Frontend compilado em bundle monolítico de 2.5 MB sem lazy-loading	frontend/src/main.tsx:1-94	Baixa
P3-LIC-01	Jurídico	**P3 - Baixo**	Preservação obrigatória de copyright e avisos da licença MIT	LICENCE.md:1-21	Baixa

---

10. AVALIAÇÃO DE DESEMPENHO E ESCALABILIDADE

10.1. Eficiência de Leitura com CQRS

O Open TMS possui uma das melhores implementações de CQRS para sistemas TypeScript:

- As telas de listagem e painéis gerenciais não realizam joins pesados em tempo de execução; elas consultam projeções desnormalizadas (`ShipmentReadModel`, `OrderReadModel`, `InvoiceReadModel`).
- Isso garante tempos de resposta inferiores a 50ms para consultas de listagem com filtros complexos.

10.2. Gargalos Estruturais Identificados

1. **Concorrência em Disco no PostgreSQL pelo `pg-boss`:**
    - O `pg-boss` utiliza tabelas internas no PostgreSQL para mensageria assíncrona. Em um cenário com centenas de empresas gerando milhares de eventos de rastreamento por minuto, a alta rotatividade de gravação e exclusão causará bloat nas tabelas e esgotará o pool de conexões do banco de dados.
2. **Armazenamento de Anexos no Banco (`BinaryStore`):**
    - O sistema possui um mecanismo de fallback para gravar anexos de até 50 MB na coluna `Bytes` da tabela `BinaryStore` caso o S3 não esteja configurado. Essa prática sobrecarrega a memória de buffer cache do PostgreSQL e o tráfego de WAL, degradando o desempenho geral do banco.
    - **Diretriz Mandatória:** Em ambiente SaaS, o MinIO ou AWS S3 / Cloudflare R2 deve ser **obrigatório**, desativando-se permanentemente o fallback em banco.

---

11. OPORTUNIDADES DE REAPROVEITAMENTO E VALOR AGREGADO

Mesmo com as lacunas identificadas, o Open TMS possui componentes extremamente avançados que devem ser preservados e reaproveitados:

1. **Motor de Embarques e Paradas (`Shipment` e `ShipmentStop`):**
    - Lógica madura para rotas com múltiplos pontos de parada, controle de janelas de coleta/entrega e tolerância a atrasos.
2. **Consolidação de Cargas LTL (Less Than Truckload):**
    - Algoritmo completo para agrupar fretes fracionados em um único manifesto de consolidação (`Consolidation`), com cálculo proporcional de receita, custo e margem esperada por trecho.
3. **WMS Operacional Completo:**
    - Mapeamento tridimensional de armazém (zonas, corredores, níveis, posições), conferência de recebimento, regras de armazenamento dirigido (putaway), divisão de ondas de separação (waves) e auditoria de embalagem.
4. **Motor de Não-Conformidades e Triage (`IssueEngine`):**
    - Sistema reativo que escuta eventos de quebra de SLA, desvio de temperatura na cadeia fria ou atraso na rota e abre ocorrências automaticamente com suporte a triagem por IA.
5. **Portais Satélites com shadcn/ui:**
    - Interfaces limpas, profissionais e responsivas para clientes, transportadoras e operadores de depósito, prontas para receber a identidade visual comercial.
6. **Campos Customizados Versionados (`CustomFields`):**
    - Permite que cada empresa cliente crie seus próprios campos em embarques e pedidos sem necessidade de alterações no schema do banco de dados.

---

12. PARECER CONCLUSIVO E PRÓXIMOS PASSOS

Parecer Conclusivo:

A base do Open TMS é **excepcional do ponto de vista arquitetural (CQRS, separação de camadas, tipagem estrita com TypeScript e Zod)** e representa uma base técnica de altíssimo nível. A decisão de utilizá-lo como acelerador tecnológico para a criação de um TMS SaaS brasileiro é **plenamente justificada e recomendada**, desde que sejam cumpridas as etapas de blindagem e adequação.

A tentativa de lançar o sistema comercialmente sem realizar as adaptações mapeadas acarretaria severos riscos de vazamento de dados entre clientes concorrentes, infração a normas de segurança e impossibilidade jurídica de emitir documentos fiscais no território nacional.

Próxima Etapa Recomendada:

Conforme a regra fundamental desta fase de auditoria, **nenhuma funcionalidade foi desenvolvida ou alterada até o momento**.

Com a conclusão desta auditoria técnica e a aprovação deste relatório, a próxima etapa imediata deve ser o início da **FASE I do Roadmap: Estabilização e Segurança**, executando a **Etapa 1 (Correções Críticas de Segurança: SSRF, Insecure Hash e Proteção de Rotas EDI)** e a **Etapa 2 (Blindagem Multi-tenant com PostgreSQL RLS e Composite Keys)**.